Deep Learning

04. 06-26

Stochastic Gradient descent: SGD

- In SGD, a random datapoint is chosen for every training step for forward and backward pass

$Z = WX$	$Z = WB_1$	$Z = WX_1$
full batch GD	minibatch GD	SGD
complete data	batch of data	single datapoint
1000	100	1

In SGD

1. Initialize weights randomly $w^{(0)}$

2. pick a single datapoint randomly from train data

$$\left.\begin{aligned} Z &= WX_1 \\ a &= f(Z) \end{aligned}\right\} \text{forward pass}$$

$$\text{loss} \longrightarrow \frac{\partial L}{\partial w^{(0)}}$$

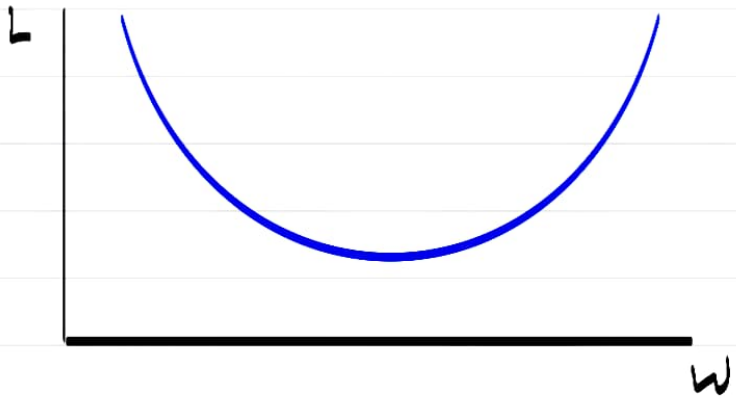
3. Update weights using gradients

$$w^{(1)} = w^{(0)} - \alpha \, \partial L / \partial w^{(0)}$$

4. Repeat step 2 and 3 untill convergence.

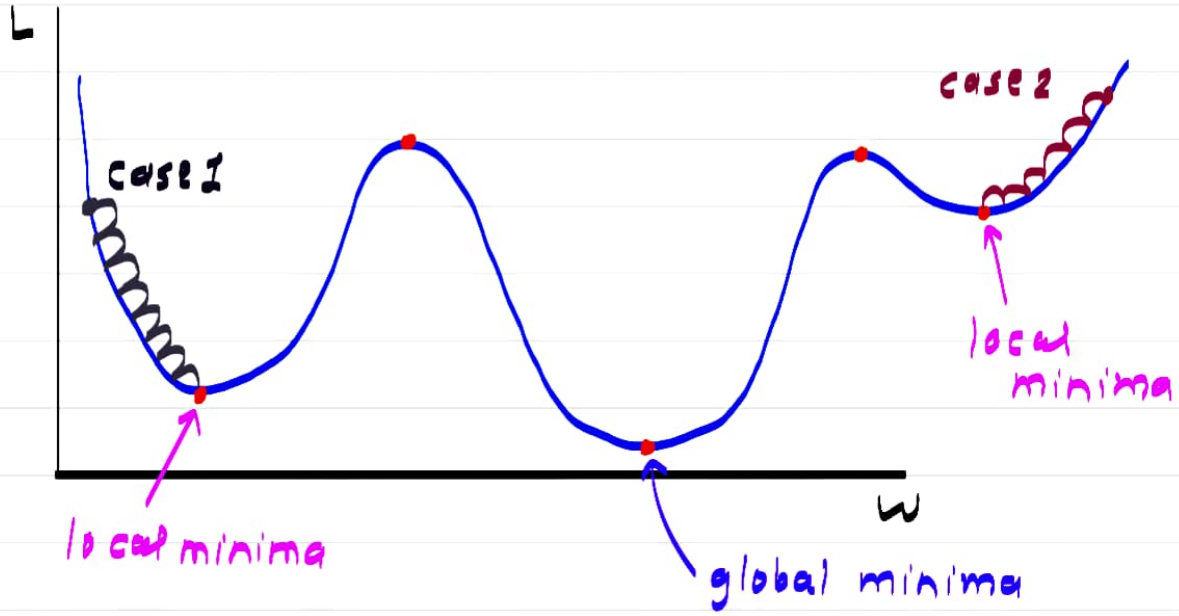
SGD is minibatch GD with batch size: 1

Getting Trapped at local minima:



→ for single perceptron

for a complex ANN



lets say our model got trained like (case 1)

- In case 1, random initialization of weights put model in such region where its near to a local minima
- Gradient descent got converged at local minima instead of global minima.

Getting converged at local minima is not always bad.
why?

1) Finding exact global minima is etremely difficult in complex ANN
but finding a local minima which gives nearly same loss as that of global minima is very easy

2) finding such low loss local minima is good for our model because- at global minima - loss is least and has higher chances of overfiting.

3) Getting converged at local minima avoids overfiting.

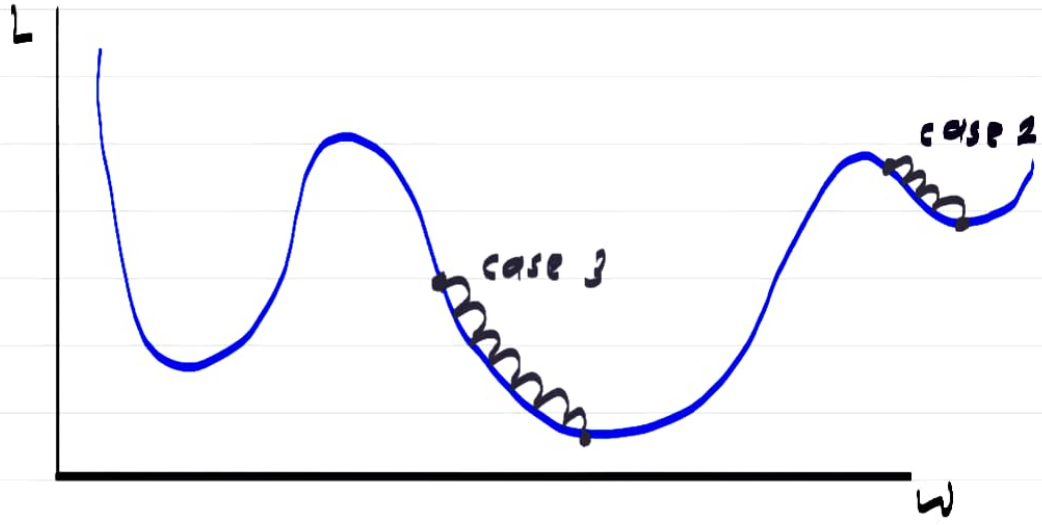
Lets say our model got trained like case 2

- model has converged at local minima, but loss is still high.

model got trapped in local minima → gradients became zero → weight updation stopped → but loss is still high → model performance is very poor

Solutions:

1) Randomly initialise weights again with different values.

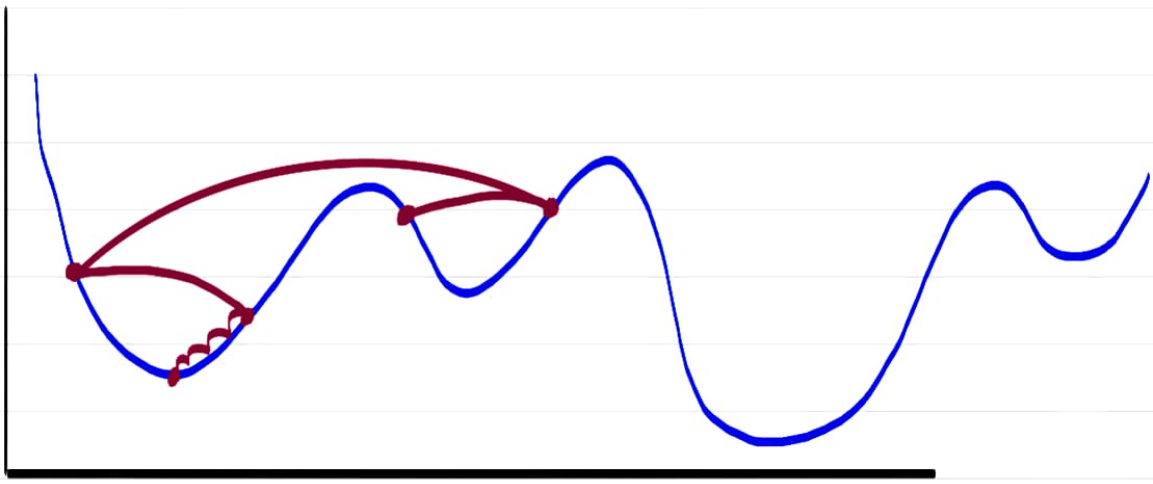


- if some case don't work out, try out different randomly initialised weights
- try and error with different regions of loss function

2) Reduce ANN complexity
- reduce no. of layers / no. of neurons / no. of input features

3) Increase learning rate - this will overshoot local minima and escape the model

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w_{old}}$$



4) Using minibatch or stochastic GD

- these add noise/randomness in model training and make model escape local minima

Optimizers in deep Learning:

- optimizers store the gradients $\partial L/\partial w$ and use them for weight updation

- they also store some other values as well which are used in model training

- they also modify the weight updation rule

Simple SGD

$$\text{gradients} = g = \partial L/\partial w_{old}$$

$$w_{new} = w_{old} - \alpha g$$

$$w_t = w_{t-1} - \alpha \, \partial L/\partial w_{t-1}$$

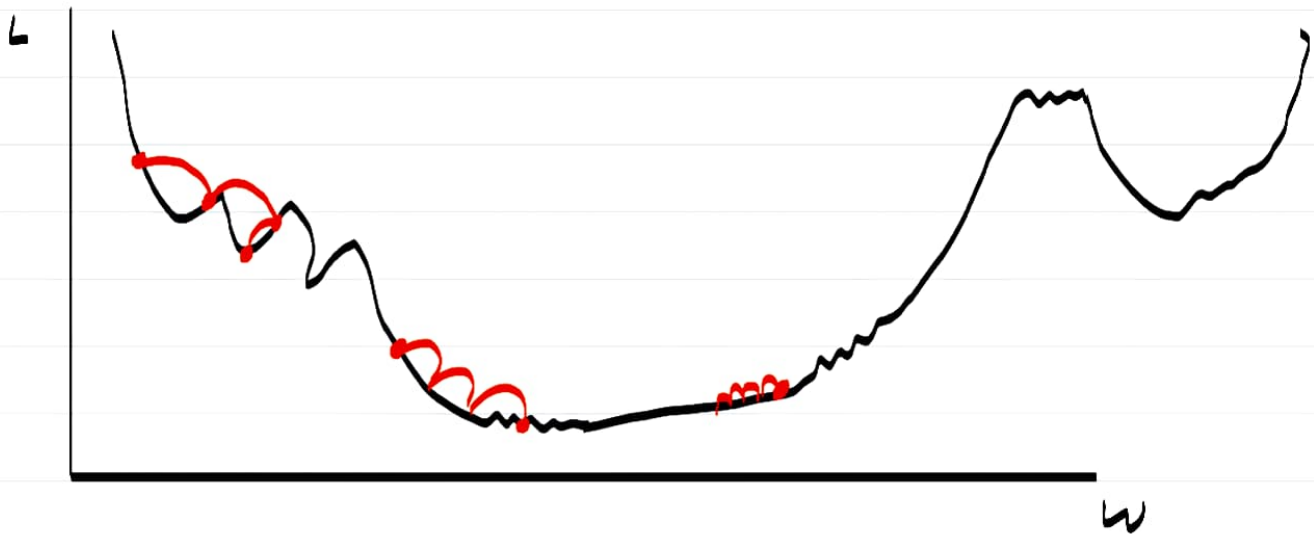
$$\boxed{w_t = w_{t-1} - \alpha g_{t-1}}$$

There are various types of optimizers which modify this weight updation formula

1) SGD with momentum :

Problem with simple SGD :

- training can become slower in some regions due to small gradients



- due to small gradients, model gets trapped near local minima
- increase in learning rate might not always help in this situations

Solution - give extra push to weight updation using momentum

1) we compute velocity of gradients in every step
2) velocity gives momentum to weight updation

new weight updation formula:

$$v_t = \gamma v_{t-1} + \alpha g_t$$

$$w_{t+1} = w_t - v_t$$

γ - momentum coefficient (0.9, 0.8, 0.85)
α - learning rate
v_t - velocity
g_t - gradient

randomly initialised weights = w_0 at $t = 0$
$\gamma = 0.9$ $\alpha = 1$

t	g_t	v_t	v_{t-1}	w
0	$\frac{\partial L}{\partial w_0} = g_0 = 2$	$v_0 = 2$	0	w_0
1	$g_1 = 1.5$	$v_1 = 3.3$	2	$w_1 = w_0 - v_0$
2	$g_2 = 0$	$v_2 = 2.97$	3.3	$w_2 = w_1 - v_1$

$v_0 = \gamma v_{0-1} + \alpha g_0$
$= 0.9 \times 0 + 1 \times 2$
$v_0 = 2$

$w_1 = w_0 - v_0$

$v_1 = \gamma v_0 + \alpha g_1$
$= 0.9 \times 2 + 1 \times 1.5$
$v_1 = 3.3$

$w_2 = w_1 - v_1$

$v_2 = \gamma v_1 + \alpha g_2$
$= 0.9 \times 3.3 + 1 \times 0$
$v_2 = 2.97$

$w_3 = w_2 - v_2$

even though gradient $g_2 = 0$, weight updation won't stop as $v_2 \neq 0$